

Dukan AI — Complete Source Code

Generated on 12 June 2026. This PDF contains all custom application code for the Dukan AI shop dashboard.

Project Dependencies

File: package.json

```
{
  "name": "tanstack_start_ts",
  "private": true,
  "sideEffects": false,
  "type": "module",
  "scripts": {
    "dev": "vite dev",
    "build": "vite build",
    "build:dev": "vite build --mode development",
    "preview": "vite preview",
    "lint": "eslint .",
    "format": "prettier --write ."
  },
  "dependencies": {
    "@hookform/resolvers": "^5.2.2",
    "@lovable.dev/cloud-auth-js": "^1.1.2",
    "@radix-ui/react-accordion": "^1.2.12",
    "@radix-ui/react-alert-dialog": "^1.1.15",
    "@radix-ui/react-aspect-ratio": "^1.1.8",
    "@radix-ui/react-avatar": "^1.1.11",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-collapsible": "^1.1.12",
    "@radix-ui/react-context-menu": "^2.2.16",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-hover-card": "^1.1.15",
    "@radix-ui/react-label": "^2.1.8",
    "@radix-ui/react-menubar": "^1.1.16",
    "@radix-ui/react-navigation-menu": "^1.2.14",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-progress": "^1.1.8",
    "@radix-ui/react-radio-group": "^1.3.8",
    "@radix-ui/react-scroll-area": "^1.2.10",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.8",
    "@radix-ui/react-slider": "^1.3.6",
    "@radix-ui/react-slot": "^1.2.4",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
    "@radix-ui/react-toggle": "^1.1.10",
    "@radix-ui/react-toggle-group": "^1.1.11",
    "@radix-ui/react-tooltip": "^1.2.8",
    "@supabase/supabase-js": "^2.108.1",
    "@tailwindcss/vite": "^4.2.1",
    "@tanstack/react-query": "^5.83.0",
    "@tanstack/react-router": "^1.168.25",
    "@tanstack/react-start": "^1.167.50",
    "@tanstack/router-plugin": "^1.167.28",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^4.1.0",
    "embla-carousel-react": "^8.6.0",
    "input-otp": "^1.4.2",
    "lucide-react": "^0.575.0",
    "react": "^19.2.0",
    "react-day-picker": "^9.14.0",
    "react-dom": "^19.2.0",
    "react-hook-form": "^7.71.2",
    "react-resizable-panels": "^4.6.5",
    "recharts": "^2.15.4",
    "sonner": "^2.0.7",
    "tailwind-merge": "^3.5.0",
    "tailwindcss": "^4.2.1",
    "tw-animate-css": "^1.3.4",
    "vaul": "^1.1.2",
    "vite-tsconfig-paths": "^6.0.2",
    "zod": "^3.24.2"
  },
  "devDependencies": {
    "@eslint/js": "^9.32.0",
    "@lovable.dev/vite-tanstack-config": "^2.3.2",
    "@types/node": "^22.16.5",
    "@types/react": "^19.2.0",
    "@types/react-dom": "^19.2.0",
    "@vitejs/plugin-react": "^5.0.4",
    "eslint": "^9.32.0",
    "eslint-config-prettier": "^10.1.1",
  }
}
```

```
"eslint-plugin-prettier": "^5.2.6",
"eslint-plugin-react-hooks": "^5.2.0",
"eslint-plugin-react-refresh": "^0.4.20",
"globals": "^15.15.0",
"nitro": "3.0.260603-beta",
"prettier": "^3.7.3",
"typescript": "^5.8.3",
"typescript-eslint": "^8.56.1",
"vite": "^7.3.1"
}
}
```

Global Styles & Theme

File: src/styles.css

```
@import "tailwindcss" source(none);
@source "../src";
@import "tw-animate-css";

@custom-variant dark (&is(.dark *));

/*
 * Design system definition.
 *
 * The @theme inline block maps CSS custom properties to Tailwind utility
 * classes (e.g. --color-primary -> bg-primary, text-primary).
 *
 * The :root and .dark blocks define the actual color values using oklch.
 * All colors MUST use oklch format.
 *
 * To add a new semantic color:
 * 1. Add the variable to :root (light value) and .dark (dark value)
 * 2. Register it in @theme inline as --color-<name>; var(--<name>);
 */

@theme inline {
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
  --radius-2xl: calc(var(--radius) + 8px);
  --radius-3xl: calc(var(--radius) + 12px);
  --radius-4xl: calc(var(--radius) + 16px);
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --color-card: var(--card);
  --color-card-foreground: var(--card-foreground);
  --color-popover: var(--popover);
  --color-popover-foreground: var(--popover-foreground);
  --color-primary: var(--primary);
  --color-primary-foreground: var(--primary-foreground);
  --color-secondary: var(--secondary);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-muted: var(--muted);
  --color-muted-foreground: var(--muted-foreground);
  --color-accent: var(--accent);
  --color-accent-foreground: var(--accent-foreground);
  --color-destructive: var(--destructive);
  --color-destructive-foreground: var(--destructive-foreground);
  --color-border: var(--border);
  --color-input: var(--input);
  --color-ring: var(--ring);
  --color-ring-offset-background: var(--background);
  --color-chart-1: var(--chart-1);
  --color-chart-2: var(--chart-2);
  --color-chart-3: var(--chart-3);
  --color-chart-4: var(--chart-4);
  --color-chart-5: var(--chart-5);
  --color-sidebar: var(--sidebar);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-ring: var(--sidebar-ring);
}

:root {
  --radius: 0.625rem;
  --background: oklch(0.99 0.01 75);
  --foreground: oklch(0.18 0.03 40);
  --card: oklch(1 0 0);
  --card-foreground: oklch(0.18 0.03 40);
  --popover: oklch(1 0 0);
  --popover-foreground: oklch(0.18 0.03 40);
  --primary: oklch(0.62 0.18 38);
  --primary-foreground: oklch(0.99 0.01 75);
  --secondary: oklch(0.95 0.03 70);
  --secondary-foreground: oklch(0.25 0.05 40);
  --muted: oklch(0.96 0.015 70);
  --muted-foreground: oklch(0.5 0.03 50);
  --accent: oklch(0.93 0.08 90);
```

```

--accent-foreground: oklch(0.25 0.06 40);
--destructive: oklch(0.58 0.22 25);
--destructive-foreground: oklch(0.984 0.003 247.858);
--border: oklch(0.9 0.02 70);
--input: oklch(0.92 0.02 70);
--ring: oklch(0.62 0.18 38);
--success: oklch(0.62 0.15 150);
--success-foreground: oklch(0.99 0.01 75);
--chart-1: oklch(0.646 0.222 41.116);
--chart-2: oklch(0.6 0.118 184.704);
--chart-3: oklch(0.398 0.07 227.392);
--chart-4: oklch(0.828 0.189 84.429);
--chart-5: oklch(0.769 0.188 70.08);
--sidebar: oklch(0.984 0.003 247.858);
--sidebar-foreground: oklch(0.129 0.042 264.695);
--sidebar-primary: oklch(0.208 0.042 265.755);
--sidebar-primary-foreground: oklch(0.984 0.003 247.858);
--sidebar-accent: oklch(0.968 0.007 247.896);
--sidebar-accent-foreground: oklch(0.208 0.042 265.755);
--sidebar-border: oklch(0.929 0.013 255.508);
--sidebar-ring: oklch(0.704 0.04 256.788);
}

.dark {
--background: oklch(0.129 0.042 264.695);
--foreground: oklch(0.984 0.003 247.858);
--card: oklch(0.208 0.042 265.755);
--card-foreground: oklch(0.984 0.003 247.858);
--popover: oklch(0.208 0.042 265.755);
--popover-foreground: oklch(0.984 0.003 247.858);
--primary: oklch(0.929 0.013 255.508);
--primary-foreground: oklch(0.208 0.042 265.755);
--secondary: oklch(0.279 0.041 260.031);
--secondary-foreground: oklch(0.984 0.003 247.858);
--muted: oklch(0.279 0.041 260.031);
--muted-foreground: oklch(0.704 0.04 256.788);
--accent: oklch(0.279 0.041 260.031);
--accent-foreground: oklch(0.984 0.003 247.858);
--destructive: oklch(0.704 0.191 22.216);
--destructive-foreground: oklch(0.984 0.003 247.858);
--border: oklch(1 0 0 / 10%);
--input: oklch(1 0 0 / 15%);
--ring: oklch(0.551 0.027 264.364);
--chart-1: oklch(0.488 0.243 264.376);
--chart-2: oklch(0.696 0.17 162.48);
--chart-3: oklch(0.769 0.188 70.08);
--chart-4: oklch(0.627 0.265 303.9);
--chart-5: oklch(0.645 0.246 16.439);
--sidebar: oklch(0.208 0.042 265.755);
--sidebar-foreground: oklch(0.984 0.003 247.858);
--sidebar-primary: oklch(0.488 0.243 264.376);
--sidebar-primary-foreground: oklch(0.984 0.003 247.858);
--sidebar-accent: oklch(0.279 0.041 260.031);
--sidebar-accent-foreground: oklch(0.984 0.003 247.858);
--sidebar-border: oklch(1 0 0 / 10%);
--sidebar-ring: oklch(0.551 0.027 264.364);
}

@layer base {
* {
border-color: var(--color-border);
}

body {
background-color: var(--color-background);
color: var(--color-foreground);
}
}

```

Router Setup

File: src/router.tsx

```
import { QueryClient } from "@tanstack/react-query";
import { createRouter } from "@tanstack/react-router";
import { routeTree } from "../routeTree.gen";

export const getRouter = () => {
  const queryClient = new QueryClient();

  const router = createRouter({
    routeTree,
    context: { queryClient },
    scrollRestoration: true,
    defaultPreloadStaleTime: 0,
  });

  return router;
};
```

Server Start Config

File: src/start.ts

```
import { createStart, createMiddleware } from "@tanstack/react-start";

import { renderErrorPage } from "../lib/error-page";
import { attachSupabaseAuth } from "@integrations/supabase/auth-attacher";

const errorMiddleware = createMiddleware().server(async ({ next }) => {
  try {
    return await next();
  } catch (error) {
    if (error != null && typeof error === "object" && "statusCode" in error) {
      throw error;
    }
    console.error(error);
    return new Response(renderErrorPage(), {
      status: 500,
      headers: { "content-type": "text/html; charset=utf-8" },
    });
  }
});

export const startInstance = createStart() => ({
  functionMiddleware: [attachSupabaseAuth],
  requestMiddleware: [errorMiddleware],
});
```

Root Layout

File: src/routes/__root.tsx

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import {
  Outlet,
  Link,
  createRootRouteWithContext,
  useRouter,
  HeadContent,
  Scripts,
} from "@tanstack/react-router";
import { useEffect, type ReactNode } from "react";

import appCss from "../styles.css?url";
import { reportLovableError } from "../lib/lovable-error-reporting";

function NotFoundComponent() {
  return (
    <div className="flex min-h-screen items-center justify-center bg-background px-4">
      <div className="max-w-md text-center">
        <h1 className="text-7xl font-bold text-foreground">404</h1>
        <h2 className="mt-4 text-xl font-semibold text-foreground">Page not found</h2>
        <p className="mt-2 text-sm text-muted-foreground">
          The page you're looking for doesn't exist or has been moved.
        </p>
        <div className="mt-6">
          <Link
            to="/"
            className="inline-flex items-center justify-center rounded-md bg-primary px-4 py-2 text-sm font-medium text-primary-foreground">
            Go home
          </Link>
        </div>
      </div>
    </div>
  );
}

function ErrorComponent({ error, reset }: { error: Error; reset: () => void }) {
  console.error(error);
  const router = useRouter();
  useEffect(() => {
    reportLovableError(error, { boundary: "tanstack_root_error_component" });
  }, [error]);

  return (
    <div className="flex min-h-screen items-center justify-center bg-background px-4">
      <div className="max-w-md text-center">
        <h1 className="text-xl font-semibold tracking-tight text-foreground">
          This page didn't load
        </h1>
        <p className="mt-2 text-sm text-muted-foreground">
          Something went wrong on our end. You can try refreshing or head back home.
        </p>
        <div className="mt-6 flex flex-wrap justify-center gap-2">
          <button
            onClick={() => {
              router.invalidate();
              reset();
            }}
            className="inline-flex items-center justify-center rounded-md bg-primary px-4 py-2 text-sm font-medium text-primary-foreground">
            Try again
          </button>
          <a
            href="/"
            className="inline-flex items-center justify-center rounded-md border border-input bg-background px-4 py-2 text-sm font-medium text-primary-foreground">
            Go home
          </a>
        </div>
      </div>
    </div>
  );
}

export const Route = createRootRouteWithContext<{ queryClient: QueryClient }>()({
  head: () => ({
    meta: [
      { charSet: "utf-8" },
    ],
  }),
});
```

```

    { name: "viewport", content: "width=device-width, initial-scale=1" },
    { title: "Lovable App" },
    { name: "description", content: "Lovable Generated Project" },
    { name: "author", content: "Lovable" },
    { property: "og:title", content: "Lovable App" },
    { property: "og:description", content: "Lovable Generated Project" },
    { property: "og:type", content: "website" },
    { name: "twitter:card", content: "summary" },
    { name: "twitter:site", content: "@Lovable" },
  ],
  links: [
    {
      rel: "stylesheet",
      href: appCss,
    },
  ],
}),
shellComponent: RootShell,
component: RootComponent,
notFoundComponent: NotFoundComponent,
errorComponent: ErrorComponent,
});

function RootShell({ children }: { children: ReactNode }) {
  return (
    <html lang="en">
      <head>
        <HeadContent />
      </head>
      <body>
        {children}
        <Scripts />
      </body>
    </html>
  );
}

function RootComponent() {
  const { queryClient } = Route.useRouteContext();

  return (
    <QueryClientProvider client={queryClient}>
      /* Required: nested routes render here. Removing <Outlet /> breaks all child routes. */
      <Outlet />
    </QueryClientProvider>
  );
}

```

Auth Page (Login / Signup / Google)

File: src/routes/auth.tsx

```
import { createFileRoute, useNavigate, Link } from "@tanstack/react-router";
import { useEffect, useState } from "react";
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
import { Store } from "lucide-react";
import { toast, Toaster } from "sonner";
import { supabase } from "@integrations/supabase/client";
import { lovable } from "@integrations/lovable/index";

export const Route = createFileRoute("/auth")({
  head: () => ({
    meta: [
      { title: "Sign in – Dukan AI" },
      { name: "description", content: "Sign in or create your Dukan AI shop account." },
    ],
  }),
  component: AuthPage,
});

function AuthPage() {
  const navigate = useNavigate();
  const [loading, setLoading] = useState(false);

  // Login state
  const [loginEmail, setLoginEmail] = useState("");
  const [loginPassword, setLoginPassword] = useState("");

  // Signup state
  const [shopName, setShopName] = useState("");
  const [ownerName, setOwnerName] = useState("");
  const [signupEmail, setSignupEmail] = useState("");
  const [signupPassword, setSignupPassword] = useState("");

  useEffect(() => {
    supabase.auth.getSession().then(({ data }) => {
      if (data.session) navigate({ to: "/" });
    });
  }, [navigate]);

  async function handleLogin(e: React.FormEvent) {
    e.preventDefault();
    setLoading(true);
    const { error } = await supabase.auth.signInWithPassword({
      email: loginEmail,
      password: loginPassword,
    });
    setLoading(false);
    if (error) {
      toast.error(error.message);
      return;
    }
    toast.success("Welcome back!");
    navigate({ to: "/" });
  }

  async function handleSignup(e: React.FormEvent) {
    e.preventDefault();
    if (!shopName.trim() || !ownerName.trim()) {
      toast.error("Please enter shop name and owner name");
      return;
    }
    setLoading(true);
    const { error } = await supabase.auth.signUp({
      email: signupEmail,
      password: signupPassword,
      options: {
        emailRedirectTo: `${window.location.origin}/`,
        data: { shop_name: shopName, owner_name: ownerName },
      },
    });
    setLoading(false);
    if (error) {
      toast.error(error.message);
      return;
    }
  }
}
```

```

    toast.success("Account created! You can sign in now.");
}

async function handleGoogle() {
    setLoading(true);
    const result = await lovable.auth.signInWithOAuth("google", {
        redirect_uri: window.location.origin,
    });
    if (result.error) {
        setLoading(false);
        toast.error("Google sign-in failed");
        return;
    }
    if (result.redirected) return;
    navigate({ to: "/" });
}

return (
    <div className="min-h-screen flex items-center justify-center bg-background px-4 py-10">
        <Toaster richColors position="top-center" />
        <div className="w-full max-w-md">
            <div className="flex flex-col items-center mb-6">
                <div className="h-12 w-12 rounded-xl bg-primary/10 grid place-items-center mb-3">
                    <Store className="h-6 w-6 text-primary" />
                </div>
                <h1 className="text-2xl font-bold">Dukan AI</h1>
                <p className="text-sm text-muted-foreground">Your shop, your dashboard</p>
            </div>

            <Card>
                <CardHeader>
                    <CardTitle>Get started</CardTitle>
                    <CardDescription>Sign in or create a new shop account.</CardDescription>
                </CardHeader>
                <CardContent>
                    <Tabs defaultValue="login">
                        <TabsList className="grid grid-cols-2 w-full">
                            <TabsTrigger value="login">Login</TabsTrigger>
                            <TabsTrigger value="signup">Sign up</TabsTrigger>
                        </TabsList>

                        <TabsContent value="login">
                            <form onSubmit={handleLogin} className="space-y-3 mt-4">
                                <div className="space-y-1">
                                    <Label htmlFor="login-email">Email</Label>
                                    <Input id="login-email" type="email" required value={loginEmail}
                                        onChange={(e) => setLoginEmail(e.target.value)} className="h-11" />
                                </div>
                                <div className="space-y-1">
                                    <Label htmlFor="login-password">Password</Label>
                                    <Input id="login-password" type="password" required value={loginPassword}
                                        onChange={(e) => setLoginPassword(e.target.value)} className="h-11" />
                                </div>
                                <Button type="submit" disabled={loading} className="w-full h-11">
                                    {loading ? "Signing in..." : "Sign in"}
                                </Button>
                            </form>
                        </TabsContent>

                        <TabsContent value="signup">
                            <form onSubmit={handleSignup} className="space-y-3 mt-4">
                                <div className="space-y-1">
                                    <Label htmlFor="shop">Shop name</Label>
                                    <Input id="shop" required value={shopName}
                                        onChange={(e) => setShopName(e.target.value)} className="h-11" />
                                </div>
                                <div className="space-y-1">
                                    <Label htmlFor="owner">Owner name</Label>
                                    <Input id="owner" required value={ownerName}
                                        onChange={(e) => setOwnerName(e.target.value)} className="h-11" />
                                </div>
                                <div className="space-y-1">
                                    <Label htmlFor="signup-email">Email</Label>
                                    <Input id="signup-email" type="email" required value={signupEmail}
                                        onChange={(e) => setSignupEmail(e.target.value)} className="h-11" />
                                </div>
                                <div className="space-y-1">
                                    <Label htmlFor="signup-password">Password (min 6 chars)</Label>
                                    <Input id="signup-password" type="password" minLength={6} required value={signupPassword}
                                        onChange={(e) => setSignupPassword(e.target.value)} className="h-11" />
                                </div>
                                <Button type="submit" disabled={loading} className="w-full h-11">
                                    {loading ? "Creating..." : "Create account"}
                                </Button>
                            </form>
                        </TabsContent>
                    </Tabs>
                </CardContent>
            </Card>
        </div>
    </div>
)

```

```

        </Button>
      </form>
    </TabsContent>
  </Tabs>

  <div className="relative my-5">
    <div className="absolute inset-0 flex items-center"><span className="w-full border-t" /></div>
    <div className="relative flex justify-center text-xs"><span className="bg-card px-2 text-muted-foreground">
      <Button type="button" variant="outline" disabled={loading} onClick={handleGoogle} className="w-full h-11">
        Continue with Google
      </Button>
    </CardContent>
  </Card>

  <p className="text-center text-xs text-muted-foreground mt-4">
    <Link to="/" className="hover:underline">Back to home</Link>
  </p>
</div>
</div>
);
}

```

Main Dashboard

File: src/routes/index.tsx

```
import { createFileRoute } from "@tanstack/react-router";
import { useEffect, useMemo, useState } from "react";
import { useNavigate } from "@tanstack/react-router";
import { supabase } from "@integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
import {
  Table, TableBody, TableCell, TableHead, TableHeader, TableRow,
} from "@components/ui/table";
import { Badge } from "@components/ui/badge";
import {
  Trash2, Plus, Store, TrendingUp, Package, IndianRupee, AlertTriangle,
  Search, BarChart3, X, Logout
} from "lucide-react";
import { toast, Toaster } from "sonner";
import {
  BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer, CartesianGrid,
} from "recharts";

export const Route = createFileRoute("/")({
  head: () => ({
    meta: [
      { title: "Dukan AI – Shop Dashboard" },
      { name: "description", content: "Manage stock, calculate sales and track profits for your shop." },
      { property: "og:title", content: "Dukan AI – Shop Dashboard" },
      { property: "og:description", content: "Manage stock, calculate sales and track profits for your shop." },
    ],
  }),
  component: Index,
});

type Product = {
  id: string;
  name: string;
  costPrice: number;
  sellPrice: number;
  stock: number;
};

type Sale = {
  id: string;
  productId: string;
  productName: string;
  qty: number;
  revenue: number;
  profit: number;
  at: number;
};

const PRODUCTS_KEY = "dukan.products";
const SALES_KEY = "dukan.sales";

const seedProducts: Product[] = [
  { id: "p1", name: "Parle-G Biscuit", costPrice: 8, sellPrice: 10, stock: 50 },
  { id: "p2", name: "Amul Milk 1L", costPrice: 58, sellPrice: 66, stock: 24 },
  { id: "p3", name: "Maggi 70g", costPrice: 12, sellPrice: 14, stock: 40 },
  { id: "p4", name: "Red Label Tea 250g", costPrice: 140, sellPrice: 160, stock: 12 },
  { id: "p5", name: "Colgate 200g", costPrice: 95, sellPrice: 110, stock: 8 },
];

function load<T>(key: string, fallback: T): T {
  if (typeof window === "undefined") return fallback;
  try {
    const raw = localStorage.getItem(key);
    return raw ? (JSON.parse(raw) as T) : fallback;
  } catch {
    return fallback;
  }
}

const inr = (n: number) =>
  new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR", maximumFractionDigits: 0 }).format(n);

function Index() {
  const navigate = useNavigate();
```

```

const [userEmail, setUserEmail] = useState<string | null>(null);
const [products, setProducts] = useState<Product[]>([]);
const [sales, setSales] = useState<Sale[]>([]);
const [hydrated, setHydrated] = useState(false);

useEffect(() => {
  const { data: sub } = supabase.auth.onAuthStateChange((_e, session) => {
    if (!session) navigate({ to: "/auth" });
    else setUserEmail(session.user.email ?? null);
  });
  supabase.auth.getSession().then(({ data }) => {
    if (!data.session) navigate({ to: "/auth" });
    else setUserEmail(data.session.user.email ?? null);
  });
  return () => sub.subscription.unsubscribe();
}, [navigate]);

useEffect(() => {
  setProducts(load(PRODUCTS_KEY, seedProducts));
  setSales(load(SALES_KEY, []));
  setHydrated(true);
}, []);

useEffect(() => {
  if (hydrated) localStorage.setItem(PRODUCTS_KEY, JSON.stringify(products));
}, [products, hydrated]);
useEffect(() => {
  if (hydrated) localStorage.setItem(SALES_KEY, JSON.stringify(sales));
}, [sales, hydrated]);

const totals = useMemo(() => {
  const revenue = sales.reduce((s, x) => s + x.revenue, 0);
  const profit = sales.reduce((s, x) => s + x.profit, 0);
  const stockValue = products.reduce((s, p) => s + p.costPrice * p.stock, 0);
  const lowStock = products.filter((p) => p.stock <= 5).length;
  return { revenue, profit, stockValue, lowStock, orders: sales.length };
}, [sales, products]);

const chartData = useMemo(() => {
  const byDay = new Map<string, number>();
  sales.forEach((s) => {
    const day = new Date(s.at).toLocaleDateString("en-IN");
    byDay.set(day, (byDay.get(day) || 0) + s.profit);
  });
  const sorted = Array.from(byDay.entries()).sort((a, b) => {
    const ad = new Date(a[0].split("/").reverse().join("-"));
    const bd = new Date(b[0].split("/").reverse().join("-"));
    return ad.getTime() - bd.getTime();
  });
  return sorted.slice(-7).map(([name, profit]) => ({ name: name.split("/").slice(0, 2).join("/"), profit }));
}, [sales]);

return (
  <div className="min-h-screen bg-background text-foreground">
    <Toaster richColors position="top-center" />
    <header className="sticky top-0 z-10 border-b bg-card/80 backdrop-blur">
      <div className="mx-auto flex max-w-6xl items-center justify-between px-4 py-3">
        <div className="flex items-center gap-3">
          <div className="flex h-9 w-9 items-center justify-center rounded-lg bg-primary text-primary-foreground shadow-sm">
            <Store className="h-4 w-4" />
          </div>
          <div>
            <h1 className="text-base font-bold leading-tight">Dukan AI</h1>
            <p className="text-[11px] text-muted-foreground">Smart shop dashboard</p>
          </div>
        </div>
      </div>
      <div className="flex items-center gap-2">
        {userEmail & & (
          <span className="hidden sm:inline text-xs text-muted-foreground max-w-[180px] truncate">
            {userEmail}
          </span>
        )}
        <Button
          size="sm"
          variant="outline"
          onClick={async () => {
            await supabase.auth.signOut();
            navigate({ to: "/auth" });
          }}
        >
          <LogOut className="h-4 w-4" />
          <span className="hidden sm:inline">Sign out</span>
        </Button>
      </div>
    </header>
  </div>

```

```

        </div>
    </div>
</header>

<main className="mx-auto max-w-6xl px-3 sm:px-4 py-4 sm:py-6 space-y-5">
    <section className="grid gap-3 sm:gap-4 grid-cols-2 lg:grid-cols-4">
        <StatCard icon={IndianRupee className="h-4 w-4"} label="Total Sales" value={inr(totals.revenue)} />
        <StatCard icon={TrendingUp className="h-4 w-4"} label="Total Profit" value={inr(totals.profit)} accent />
        <StatCard icon={Package className="h-4 w-4"} label="Stock Value" value={inr(totals.stockValue)} />
        <StatCard icon={AlertTriangle className="h-4 w-4"} label="Low Stock" value={String(totals.lowStock)} />
    </section>

    {chartData.length > 0 & & (
        <Card>
            <CardHeader className="pb-2">
                <CardTitle className="text-sm flex items-center gap-2">
                    <BarChart3 className="h-4 w-4 text-primary" />
                    Daily Profit (Last 7 Days)
                </CardTitle>
            </CardHeader>
            <CardContent>
                <div className="h-48 w-full">
                    <ResponsiveContainer width="100%" height="100%">
                        <BarChart data={chartData}>
                            <CartesianGrid strokeDasharray="3 3" stroke="var(--border)" />
                            <XAxis dataKey="name" tick={{ fontSize: 11 }} />
                            <YAxis tickFormatter={(v) => `₹${v}`} tick={{ fontSize: 11 }} width={50} />
                            <Tooltip formatter={(v: number) => inr(v)} />
                            <Bar dataKey="profit" fill="var(--primary)" radius={[4, 4, 0, 0]} />
                        </BarChart>
                    </ResponsiveContainer>
                </div>
            </CardContent>
        </Card>
    )}

    <Tabs defaultValue="sell" className="w-full">
        <TabsList className="grid w-full grid-cols-3 h-11">
            <TabsTrigger value="sell" className="text-sm">Sell</TabsTrigger>
            <TabsTrigger value="stock" className="text-sm">Stock</TabsTrigger>
            <TabsTrigger value="history" className="text-sm">History</TabsTrigger>
        </TabsList>

        <TabsContent value="sell" className="mt-3">
            <SellCalculator
                products={products}
                onSell={(productId, qty) => {
                    const p = products.find((x) => x.id === productId);
                    if (!p) return;
                    if (qty <= 0) return toast.error("Quantity must be > 0");
                    if (qty > p.stock) return toast.error(`Only ${p.stock} in stock`);
                    const revenue = p.sellPrice * qty;
                    const profit = (p.sellPrice - p.costPrice) * qty;
                    setProducts((prev) => {
                        prev.map((x) => (x.id === p.id ? { ...x, stock: x.stock - qty } : x));
                    });
                    setSales((prev) => [
                        { id: crypto.randomUUID(), productId: p.id, productName: p.name, qty, revenue, profit, at: Date.now() },
                        ...prev,
                    ]);
                    toast.success(`Sold ${qty} × ${p.name} – profit ${inr(profit)}`);
                }}
            />
        </TabsContent>

        <TabsContent value="stock" className="mt-3">
            <StockManager
                products={products}
                onAdd={(p) => setProducts((prev) => [...prev, p])}
                onRemove={(id) => setProducts((prev) => prev.filter((x) => x.id !== id))}
                onRestock={(id, qty) => {
                    setProducts((prev) => {
                        prev.map((x) => (x.id === id ? { ...x, stock: x.stock + qty } : x));
                    });
                }}
            />
        </TabsContent>

        <TabsContent value="history" className="mt-3">
            <SalesHistory sales={sales} onClear={() => setSales([])} />
        </TabsContent>
    </Tabs>
</main>
</div>
);

```

```

}

function StatCard({ icon, label, value, accent }: { icon: React.ReactNode; label: string; value: string; accent?: boolean }) {
  return (
    <Card className={accent ? "border-primary/30 bg-accent/40" : ""}>
      <CardContent className="p-3 sm:p-4">
        <div className="flex items-center justify-between text-muted-foreground">
          <span className="text-[10px] sm:text-xs font-medium uppercase tracking-wide">{label}</span>
          <span className="text-primary">{icon}</span>
        </div>
        <div className="mt-1 sm:mt-2 text-xl sm:text-2xl font-bold tabular-nums">{value}</div>
      </CardContent>
    </Card>
  );
}

function SellCalculator({ products, onSell }: { products: Product[]; onSell: (id: string, qty: number) => void }) {
  const [productId, setProductId] = useState<string>("");
  const [qty, setQty] = useState<number>(1);
  const p = products.find((x) => x.id === productId);
  const revenue = p ? p.sellPrice * qty : 0;
  const profit = p ? (p.sellPrice - p.costPrice) * qty : 0;

  return (
    <Card>
      <CardHeader className="pb-2">
        <CardTitle className="text-sm sm:text-base">Quick Sale Calculator</CardTitle>
      </CardHeader>
      <CardContent className="space-y-4">
        {products.length === 0 ? (
          <p className="text-sm text-muted-foreground">Add products in the Stock tab first.</p>
        ) : (
          <div className="grid gap-3">
            <div className="space-y-1.5">
              <Label className="text-xs sm:text-sm">Product</Label>
              <select
                value={productId}
                onChange={(e) => setProductId(e.target.value)}
                className="flex h-11 w-full rounded-md border bg-background px-3 text-sm">
                <option value="">Select product...</option>
                {products.map((x) => (
                  <option key={x.id} value={x.id}>
                    {x.name} - {inr(x.sellPrice)} ({x.stock} left)
                  </option>
                ))}
              </select>
            </div>
            <div className="space-y-1.5">
              <Label className="text-xs sm:text-sm">Quantity</Label>
              <div className="flex items-center gap-2">
                <button
                  className="h-11 w-11 rounded-md border bg-muted/40 text-lg font-bold active:bg-muted"
                  onClick={() => setQty((q) => Math.max(1, q - 1))}
                  type="button">-</button>
                <input
                  type="number"
                  min={1}
                  value={qty}
                  onChange={(e) => setQty(Math.max(1, Number(e.target.value) || 1))}
                  className="text-center text-lg font-semibold h-11">
                <button
                  className="h-11 w-11 rounded-md border bg-muted/40 text-lg font-bold active:bg-muted"
                  onClick={() => setQty((q) => q + 1)}
                  type="button">+</button>
              </div>
            </div>
          </div>
        )}
        <div className="grid gap-2 grid-cols-3">
          <Mini label="Revenue" value={inr(revenue)} />
          <Mini label="Profit" value={inr(profit)} highlight />
          <Mini label="Left after" value={p ? String(Math.max(0, p.stock - qty)) : "-"} />
        </div>
      </CardContent>
    </Card>
  );
}

```

```

        size="lg"
        className="w-full h-12 text-base"
        disabled={!productId}
        onClick={() => {
          onSell(productId, qty);
          setQty(1);
        }}
      >
        Record Sale
      </Button>
    </div>
  )}
  </CardContent>
</Card>
);
}

function Mini({ label, value, highlight }: { label: string; value: string; highlight?: boolean }) {
  return (
    <div className={`rounded-lg border p-2 sm:p-3 text-center ${highlight ? "bg-accent/50 border-primary/30" : "bg-muted/30"}
      <div className="text-[10px] sm:text-xs text-muted-foreground">{label}</div>
      <div className="text-sm sm:text-lg font-semibold tabular-nums">{value}</div>
    </div>
  );
}

function StockManager({
  products,
  onAdd,
  onRemove,
  onRestock,
}: {
  products: Product[];
  onAdd: (p: Product) => void;
  onRemove: (id: string) => void;
  onRestock: (id: string, qty: number) => void;
}) {
  const [name, setName] = useState("");
  const [cost, setCost] = useState<number>(0);
  const [sell, setSell] = useState<number>(0);
  const [stock, setStock] = useState<number>(0);
  const [search, setSearch] = useState("");
  const filtered = useMemo(() => {
    const q = search.trim().toLowerCase();
    return q ? products.filter((p) => p.name.toLowerCase().includes(q)) : products;
  }, [products, search]);

  return (
    <div className="space-y-4">
      <Card>
        <CardHeader className="pb-2">
          <CardTitle className="text-sm sm:text-base">Add Product</CardTitle>
        </CardHeader>
        <CardContent className="space-y-3">
          <div className="space-y-1.5">
            <Label className="text-xs sm:text-sm">Name</Label>
            <Input value={name} onChange={(e) => setName(e.target.value)} placeholder="e.g. Tata Salt 1kg" />
          </div>
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-1.5">
              <Label className="text-xs sm:text-sm">Cost ■</Label>
              <Input type="number" min={0} value={cost} onChange={(e) => setCost(Number(e.target.value) || 0)} />
            </div>
            <div className="space-y-1.5">
              <Label className="text-xs sm:text-sm">Sell ■</Label>
              <Input type="number" min={0} value={sell} onChange={(e) => setSell(Number(e.target.value) || 0)} />
            </div>
          </div>
          <div className="space-y-1.5">
            <Label className="text-xs sm:text-sm">Initial Stock</Label>
            <Input type="number" min={0} value={stock} onChange={(e) => setStock(Number(e.target.value) || 0)} />
          </div>
          <Button
            className="w-full"
            onClick={() => {
              if (!name.trim()) return toast.error("Name is required");
              if (sell < cost) return toast.error("Sell price should be ≥ cost");
              onAdd({ id: crypto.randomUUID(), name: name.trim(), costPrice: cost, sellPrice: sell, stock });
              setName(""); setCost(0); setSell(0); setStock(0);
              toast.success("Product added");
            }}
          >
            <Plus className="mr-2 h-4 w-4" /> Add to Stock
          </Button>
        </CardContent>
      </Card>
    </div>
  );
}

```

```

</Card>;
<CardHeader className="pb-2 flex flex-row items-center justify-between gap-2">;
  <CardTitle className="text-sm sm:text-base">Inventory</CardTitle>;
  <Badge variant="secondary">{filtered.length}</Badge>;
</CardHeader>;
<CardContent className="space-y-3">;
  <div className="relative">;
    <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />;
    <Input
      value={search}
      onChange={(e) => setSearch(e.target.value)}
      placeholder="Search products..."
      className="pl-9"
    />;
    {search & & (
      <button
        className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground"
        onClick={() => setSearch("")}
      >;
      <X className="h-4 w-4" />;
      </button>;
    )}
  </div>;

{filtered.length === 0 ? (
  <p className="text-sm text-muted-foreground text-center py-6">No products found.</p>;
) : (
  <div className="space-y-2">;
    <div /* Desktop table */
      <div className="hidden sm:block overflow-x-auto">;
        <Table>;
          <TableHeader>;
            <TableRow>;
              <TableHead>Product</TableHead>;
              <TableHead className="text-right">Cost</TableHead>;
              <TableHead className="text-right">Sell</TableHead>;
              <TableHead className="text-right">Margin</TableHead>;
              <TableHead className="text-right">Stock</TableHead>;
              <TableHead className="text-right">Actions</TableHead>;
            </TableRow>;
          </TableHeader>;
          <TableBody>;
            {filtered.map((p) => (
              <TableRow key={p.id}>;
                <TableCell className="font-medium">{p.name}</TableCell>;
                <TableCell className="text-right tabular-nums">{inr(p.costPrice)}</TableCell>;
                <TableCell className="text-right tabular-nums">{inr(p.sellPrice)}</TableCell>;
                <TableCell className="text-right tabular-nums">{inr(p.sellPrice - p.costPrice)}</TableCell>;
                <TableCell className="text-right">
                  {p.stock &= 5 ? (
                    <Badge variant="destructive">{p.stock}</Badge>;
                  ) : (
                    <span className="tabular-nums">{p.stock}</span>;
                  )}
                </TableCell>;
                <TableCell className="text-right">
                  <div className="flex justify-end gap-1">
                    <Button size="sm" variant="outline" onClick={() => onRestock(p.id, 10)}>+10</Button>;
                    <Button size="sm" variant="ghost" onClick={() => onRemove(p.id)}></Button>;
                    <Trash2 className="h-4 w-4" />;
                  </div>
                </TableCell>;
              </TableRow>;
            )})}
          </TableBody>;
        </Table>;
      </div>;
    </div>;

    <div /* Mobile cards */
      <div className="sm:hidden space-y-2">;
        {filtered.map((p) => (
          <div key={p.id} className="rounded-lg border p-3 space-y-2 bg-card">;
            <div className="flex items-center justify-between">
              <span className="font-medium text-sm">{p.name}</span>;
              {p.stock &= 5 ? (
                <Badge variant="destructive" className="text-[10px]">Low: {p.stock}</Badge>;
              ) : (
                <Badge variant="secondary" className="text-[10px]">Stock: {p.stock}</Badge>;
              )}
            </div>;
            <div className="grid grid-cols-3 gap-2 text-xs text-muted-foreground">
              <div>Cost: <span className="text-foreground font-medium">{inr(p.costPrice)}</span></div>
            </div>
          </div>
        )})}
      </div>
    </div>
  )}

```

[illegible]

```

                </div>;
            </div>;
        )}
        </div>;
        </div>;
    )}
    </CardContent>;
    </Card>;
);
}

```

Supabase Browser Client

File: src/integrations/supabase/client.ts

```
// This file is automatically generated. Do not edit it directly.
import { createClient } from '@supabase/supabase-js';
import type { Database } from './types';

function createSupabaseClient() {
  // Use import.meta.env for client-side (Vite build-time replacement)
  // Fall back to process.env for SSR (server-side rendering)
  const SUPABASE_URL = import.meta.env.VITE_SUPABASE_URL || process.env.SUPABASE_URL;
  const SUPABASE_PUBLISHABLE_KEY = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY || process.env.SUPABASE_PUBLISHABLE_KEY;

  if (!SUPABASE_URL || !SUPABASE_PUBLISHABLE_KEY) {
    const missing = [
      ...(!SUPABASE_URL ? ['SUPABASE_URL'] : []),
      ...(!SUPABASE_PUBLISHABLE_KEY ? ['SUPABASE_PUBLISHABLE_KEY'] : []),
    ];
    const message = `Missing Supabase environment variable(s): ${missing.join(', ')}. Connect Supabase in Lovable Cloud.`;
    console.error(`[Supabase] ${message}`);
    throw new Error(message);
  }

  return createClient<Database>(SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, {
    auth: {
      storage: typeof window !== 'undefined' ? localStorage : undefined,
      persistSession: true,
      autoRefreshToken: true,
    },
  });
}

let _supabase: ReturnType<typeof createSupabaseClient> | undefined;

// Import the supabase client like this:
// import { supabase } from "@integrations/supabase/client";
export const supabase = new Proxy({} as ReturnType<typeof createSupabaseClient>, {
  get(_, prop, receiver) {
    if (!_supabase) _supabase = createSupabaseClient();
    return Reflect.get(_supabase, prop, receiver);
  },
});
```

Auth Middleware

File: src/integrations/supabase/auth-middleware.ts

```
// This file is automatically generated. Do not edit it directly.
import { createMiddleware } from '@tanstack/react-start'
import { getRequest } from '@tanstack/react-start/server'
import { createClient } from 'supabase/supabase-js'
import type { Database } from './types'

export const requireSupabaseAuth = createMiddleware({ type: 'function' }).server(
  async ({ next }) => {

    const SUPABASE_URL = process.env.SUPABASE_URL;
    const SUPABASE_PUBLISHABLE_KEY = process.env.SUPABASE_PUBLISHABLE_KEY;

    if (!SUPABASE_URL || !SUPABASE_PUBLISHABLE_KEY) {
      const missing = [
        ...(!SUPABASE_URL ? ['SUPABASE_URL'] : []),
        ...(!SUPABASE_PUBLISHABLE_KEY ? ['SUPABASE_PUBLISHABLE_KEY'] : []),
      ];
      const message = `Missing Supabase environment variable(s): ${missing.join(', ')}. Connect Supabase in Lovable Cloud.`;
      console.error(`[Supabase] ${message}`);
      throw new Error(message);
    }

    const request = getRequest();

    if (!request?.headers) {
      throw new Error('Unauthorized: No request headers available');
    }

    const authHeader = request.headers.get('authorization');

    if (!authHeader) {
      throw new Error('Unauthorized: No authorization header provided');
    }

    if (!authHeader.startsWith('Bearer ')) {
      throw new Error('Unauthorized: Only Bearer tokens are supported');
    }

    const token = authHeader.replace('Bearer ', '');
    if (!token) {
      throw new Error('Unauthorized: No token provided');
    }

    const supabase = createClient<Database>({
      SUPABASE_URL!,
      SUPABASE_PUBLISHABLE_KEY!,
    }, {
      global: {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      },
      auth: {
        storage: undefined,
        persistSession: false,
        autoRefreshToken: false,
      },
    });

    const { data, error } = await supabase.auth.getClaims(token);
    if (error || !data?.claims) {
      throw new Error('Unauthorized: Invalid token');
    }

    if (!data.claims.sub) {
      throw new Error('Unauthorized: No user ID found in token');
    }

    return next({
      context: {
        supabase,
        userId: data.claims.sub,
        claims: data.claims,
      },
    });
  });
```


Auth Attacher

File: src/integrations/supabase/auth-attacher.ts

```
// This file is automatically generated. Do not edit it directly.
import { createMiddleware } from '@tanstack/react-start'
import { supabase } from './client'

// Must be registered as a global `functionMiddleware` in `src/start.ts`; otherwise
// the browser never attaches the bearer token to serverFn RPCs.
export const attachSupabaseAuth = createMiddleware({ type: 'function' }).client(
  async ({ next }) => {
    const { data } = await supabase.auth.getSession()
    const token = data.session?.access_token
    return next({
      headers: token ? { Authorization: `Bearer ${token}` } : {},
    })
  },
)
```

Lovable Auth Integration

File: src/integrations/lovable/index.ts

// This file is auto-generated by Lovable. Do not modify it.

```
import { createLovableAuth } from "@lovable.dev/cloud-auth-js";
import { supabase } from "../supabase/client";
const lovableAuth = createLovableAuth();

type SignInOptions = {
  redirect_uri?: string;
  extraParams?: Record<string, string>;
};

export const lovable = {
  auth: {
    signInWithOAuth: async (provider: "google" | "apple" | "microsoft" | "lovable", opts?: SignInOptions) => {
      const result = await lovableAuth.signInWithOAuth(provider, {
        redirect_uri: opts?.redirect_uri,
        extraParams: {
          ...opts?.extraParams,
        },
      });
      if (result.redirected) {
        return result;
      }
      if (result.error) {
        return result;
      }
      try {
        await supabase.auth.setSession(result.tokens);
      } catch (e) {
        return { error: e instanceof Error ? e : new Error(String(e)) };
      }
      return result;
    },
  },
};
```

DB Migration — 20260612110915_789430f3-4da5-4073-a32b-b60751ac1fa1.sql

File: supabase/migrations/20260612110915_789430f3-4da5-4073-a32b-b60751ac1fa1.sql

```
CREATE TABLE public.profiles (  
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  shop_name TEXT,  
  owner_name TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
GRANT SELECT, INSERT, UPDATE, DELETE ON public.profiles TO authenticated;  
GRANT ALL ON public.profiles TO service_role;  
  
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;  
  
CREATE POLICY "Users view own profile" ON public.profiles FOR SELECT TO authenticated USING (auth.uid() = id);  
CREATE POLICY "Users insert own profile" ON public.profiles FOR INSERT TO authenticated WITH CHECK (auth.uid() = id);  
CREATE POLICY "Users update own profile" ON public.profiles FOR UPDATE TO authenticated USING (auth.uid() = id) WITH CHECK (auth.  
  
CREATE OR REPLACE FUNCTION public.handle_new_user()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
SECURITY DEFINER  
SET search_path = public  
AS $$  
BEGIN  
  INSERT INTO public.profiles (id, shop_name, owner_name)  
  VALUES (  
    NEW.id,  
    NEW.raw_user_meta_data->&'shop_name',  
    NEW.raw_user_meta_data->&'owner_name'  
  );  
  RETURN NEW;  
END;  
$$;  
  
CREATE TRIGGER on_auth_user_created  
AFTER INSERT ON auth.users  
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

DB Migration — 20260612110942_b0c9a3f9-69cf-4e60-a5eb-aff2de34007e.sql

File: supabase/migrations/20260612110942_b0c9a3f9-69cf-4e60-a5eb-aff2de34007e.sql

```
REVOKE EXECUTE ON FUNCTION public.handle_new_user() FROM PUBLIC, anon, authenticated;
```

UI Components (shadcn/ui)

The following standard shadcn/ui components are used and live under `src/components/ui/`. They are auto-generated by the shadcn CLI and not included in full here to keep the PDF readable: accordion, alert, alert-dialog, aspect-ratio, avatar, badge, breadcrumb, button, calendar, card, carousel, chart, checkbox, collapsible, command, context-menu, dialog, drawer, dropdown-menu, form, hover-card, input, input-otp, label, menubar, navigation-menu, pagination, popover, progress, radio-group, resizable, scroll-area, select, separator, sheet, sidebar, skeleton, slider, sonner, switch, table, tabs, textarea, toggle, toggle-group, tooltip.

Project Metadata

Framework: TanStack Start v1 + React 19 + Vite 7

Styling: Tailwind CSS v4 + shadcn/ui

Backend: Lovable Cloud (Supabase) — Auth + Postgres

Auth methods: Email/password + Google OAuth

Charts: Recharts

Toasts: Sonner